

GOOGLE CLOUD PLATFORM SECURITY LAB

Part 4 – Application Load Balancer using Google Compute Engine

Submitted By

Name: Senthamizhan J

Domain : Cloud Security Engineering

Platform : Google Cloud Platform

Table of Contents

1. Introduction
2. Objectives
3. Architecture
4. Prerequisites
5. Implementation
6. Results
7. Conclusion

1. Introduction

Google Cloud Application Load Balancer distributes incoming HTTP requests across multiple backend virtual machines, ensuring high availability, scalability, and fault tolerance. In this lab, multiple Compute Engine instances were deployed, grouped into a Managed Instance Group, monitored using Health Checks, and exposed through an External HTTP Load Balancer.

2. Objectives

The objectives of this lab are:

- Configure Google Cloud environment
- Deploy multiple Compute Engine virtual machines
- Configure firewall rules
- Create an Instance Template
- Deploy a Managed Instance Group

- Configure HTTP Health Checks
- Create a Backend Service
- Configure URL Mapping
- Configure Target HTTP Proxy
- Create Global Forwarding Rule
- Verify Load Balancer functionality

3. System Architecture

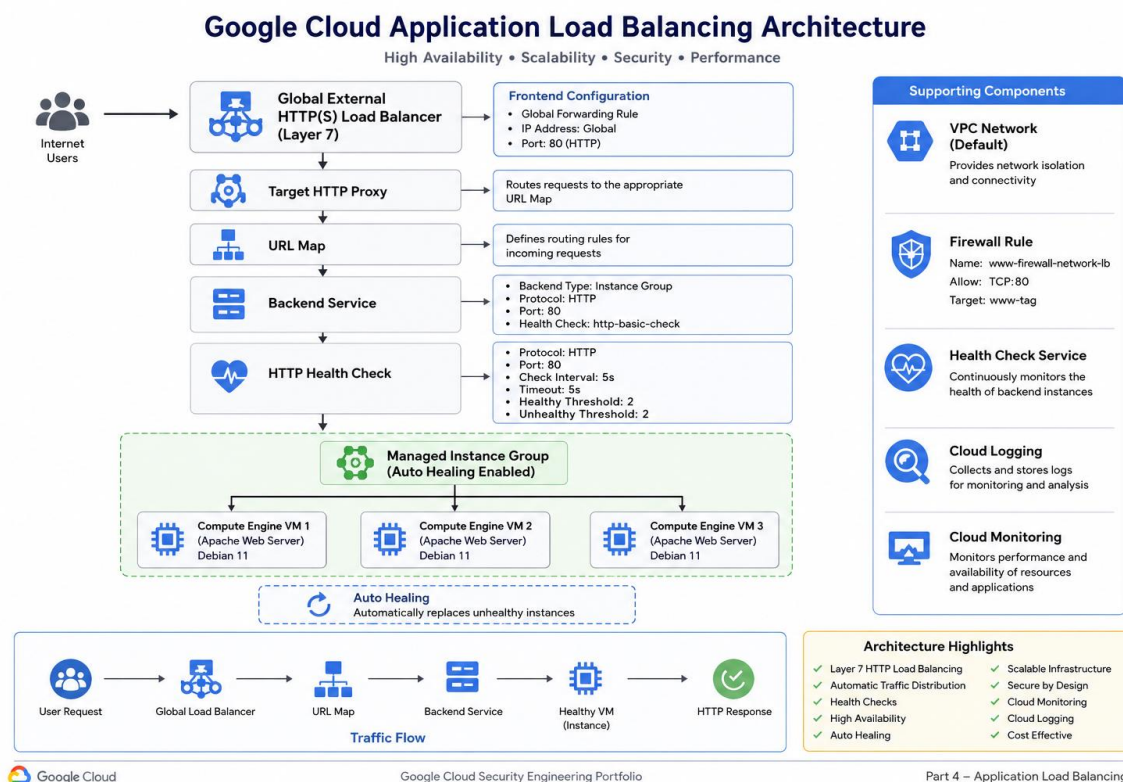


Figure 1. Google Cloud Application Load Balancer Architecture

4. Prerequisites

Before beginning the implementation, the following services were available:

- Google Cloud Project
- Google Cloud Shell
- Compute Engine API

- External Network Access
- Google Cloud Console
- gcloud CLI

5. Implementation

Step 1 – Configure Region and Zone

The default region and zone were configured for deploying all cloud resources.

```

Welcome to Cloud Shell! Type "help" to get started, or type "gcpinfo" to try prompting with Shell! CLI.
To set your Cloud Platform project in this session, use "gcloud config set project [PROJECT_ID]".
You can view your projects by running "gcloud projects list".
gcloudconfig@_20a7067c1aeb1c0db0a11:- gcloud config set project gcpk1ab0-gcp-04-83f0c14ba20f
Warning: [WARNING] Read more to tag: gcpk1ab0-gcp-04-83f0c14ba20f
Updated property [compute/region]:
student_01_20a7067c1aeb1c0db0a11:- [gcpk1ab0-gcp-04-83f0c14ba20f] gcloud config set compute/region europe-west4
Updated property [compute/region]:
student_01_20a7067c1aeb1c0db0a11:- [gcpk1ab0-gcp-04-83f0c14ba20f]

```

Figure 2. Configuring Default Region and Zone

Step 2 – Create Compute Engine Virtual Machines

Three Apache web server virtual machines were created.

- www1
- www2
- www3

```

www
$ curl -v http://www.gcpcloud.com/compute/v1/projects/gcpk1ab0-gcp-04-83f0c14ba20f/zones/europe-west-4/instances/www1.
NAME: www1
ZONE: europe-west-4-a
MACHINE_TYPE: n1-standard-1
PRIORITY: standard
EXTERNAL_IP: 10.164.2.4
EXTERNAL_IP_A: 34.30.128.113
STATUS: RUNNING
student_01_20a7067c1aeb1c0db0a11:- [gcpk1ab0-gcp-04-83f0c14ba20f] gcloud compute instances list
NAME: www1
ZONE: europe-west-4-a
MACHINE_TYPE: n1-standard-1
PRIORITY: standard
EXTERNAL_IP: 10.164.2.4
EXTERNAL_IP_A: 34.30.128.113
STATUS: RUNNING
NAME: www2
ZONE: europe-west-4-a
MACHINE_TYPE: n1-standard-1
PRIORITY: standard
EXTERNAL_IP: 10.164.2.3
EXTERNAL_IP_A: 34.30.128.114
STATUS: RUNNING
NAME: www3
ZONE: europe-west-4-a
MACHINE_TYPE: n1-standard-1
PRIORITY: standard
EXTERNAL_IP: 10.164.2.4
EXTERNAL_IP_A: 34.30.128.115
STATUS: RUNNING
student_01_20a7067c1aeb1c0db0a11:- [gcpk1ab0-gcp-04-83f0c14ba20f]

```

Figure 3. Compute Engine Instances

Step 3 – Configure Firewall Rule

A firewall rule allowing HTTP traffic on port 80 was created.

```
NAME: www
ZONE: europe-west4-b
MACHINE_TYPE: e2-medium
PRIMARY_IP: 10.144.0.3
INTERNAL_IP: 10.144.0.3
EXTERNAL_IP: 34.30.124.115
STATUS: RUNNING

NAME: www2
ZONE: europe-west4-b
MACHINE_TYPE: e2-medium
PRIMARY_IP: 10.144.0.4
INTERNAL_IP: 10.144.0.4
EXTERNAL_IP: 34.30.124.115
STATUS: RUNNING

student_01_2063f087c1aeb@cloudshell:~$ (qwiklabs-gcp-04-83f06148e20f) gcloud compute firewall-rules create www-firewall-network-lb \
--target-tags network-lb-tag --allow tcp:80
Creating Firewall...working...Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-04-83f06148e20f/global/firewalls/www-firewall-network-lb].
Creating Firewall...done.
NAME: www-firewall-network-lb
NETWORK: default
DIRECTION: INGRESS
PRIORITY: 1000
ALLOW: tcp:80
DENY:
DISABLED: False
student_01_2063f087c1aeb@cloudshell:~$ (qwiklabs-gcp-04-83f06148e20f) |
```

Figure 4. Firewall Rule Creation

Step 4 – Create Instance Template

An Instance Template was created to standardize the configuration of backend instances.

```
student_01_2063f087c1aeb@cloudshell:~$ (qwiklabs-gcp-04-83f06148e20f) gcloud compute instance-templates create lb-backend-template \
--region europe-west4 \
--network=default \
--subnet=default \
--tags=allow-health-check \
--shield=type=medium \
--image-family=debian-11 \
--image-project=debian-cloud \
--metadata=metadata=cloud \
--metadata=metadata=cloud \
--metadata=startup-script='#!/bin/bash
apt-get update
apt-get install apache2 -y
a2enmod ssl
a2enmod ssl
vm_hostname=$(curl -s "Metadata-Flavor:Google" |
http://169.254.169.254/computeMetadata/v1/instance/name)
echo "Page served from: $vm_hostname" | \
tee /var/www/html/index.html
systemctl restart apache2'
Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-04-83f06148e20f/global/instanceTemplates/lb-backend-template].
NAME: lb-backend-template
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
CREATION_TIMESTAMP: 2024-06-25T10:07:31.894-07:00
student_01_2063f087c1aeb@cloudshell:~$ (qwiklabs-gcp-04-83f06148e20f) |
```

Figure 5. Instance Template

Step 5 – Create Managed Instance Group

A Managed Instance Group containing backend virtual machines was created.

```
tee /var/www/html/index.html
systemctl restart apache2
Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-04-83f06148e20f/global/instanceTemplates/lb-backend-template].
NAME: lb-backend-template
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
CREATION_TIMESTAMP: 2024-06-25T10:07:31.894-07:00
student_01_2063f087c1aeb@cloudshell:~$ (qwiklabs-gcp-04-83f06148e20f) gcloud compute instance-groups managed create lb-backend-group \
--template=lb-backend-template --size=2 --zone=europe-west4-b
Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-04-83f06148e20f/zones/europe-west4-b/instanceGroupsManaged/lb-backend-group].
NAME: lb-backend-group
LOCATION: europe-west4-b
SOURCE: zone
BACK_INSTANCE_NAME: lb-backend-group
SIZE: 2
TARGET_SIZE: 2
INSTANCE_TEMPLATE: lb-backend-template
AUTOSCALING: no
student_01_2063f087c1aeb@cloudshell:~$ (qwiklabs-gcp-04-83f06148e20f) gcloud compute firewall-rules create fw-allow-health-check \
--network=default \
--action=allow \
--direction=ingress \
--source-ranges=10.211.0.0/22,35.191.0.0/16 \
--target-tags=allow-health-check \
--rules-top:80
Creating Firewall...working...Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-04-83f06148e20f/global/firewalls/fw-allow-health-check].
Creating Firewall...done.
NAME: fw-allow-health-check
NETWORK: default
DIRECTION: INGRESS
PRIORITY: 1000
ALLOW: tcp:80
DENY:
DISABLED: False
student_01_2063f087c1aeb@cloudshell:~$ (qwiklabs-gcp-04-83f06148e20f) |
```

Figure 6. Managed Instance Group

Step 6 – Configure HTTP Health Check

A Health Check was configured to monitor backend instance availability.

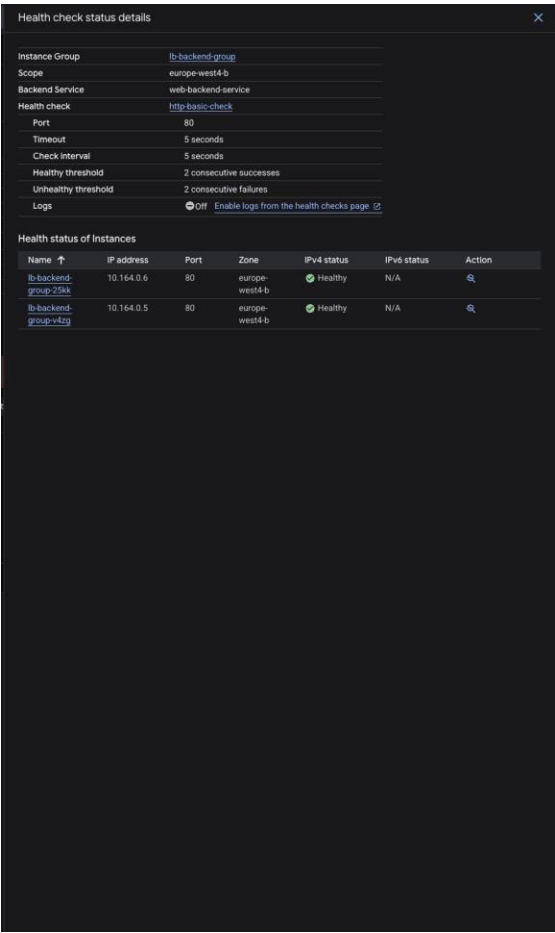


Figure 7. HTTP Health Check

Step 7 – Configure Backend Service

The Managed Instance Group was attached to the Backend Service.

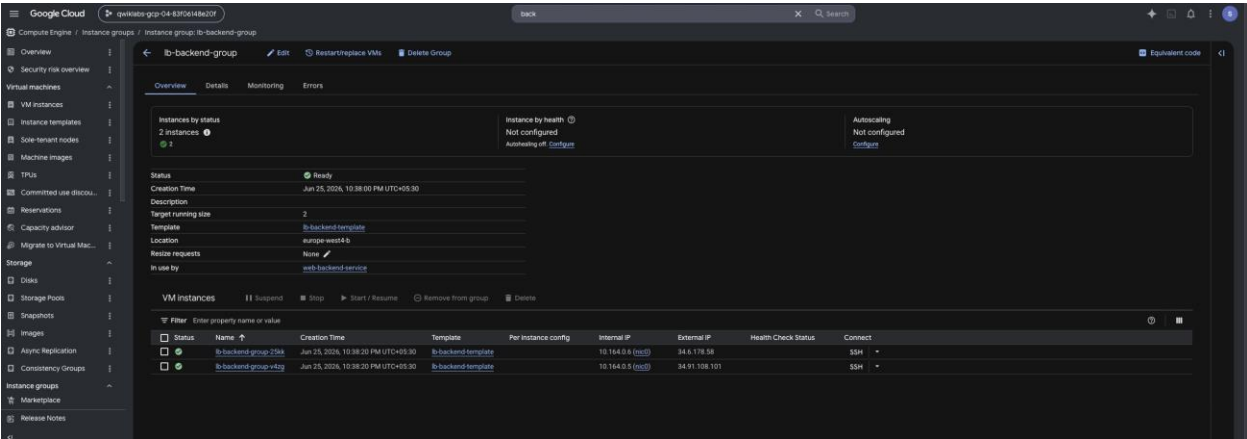
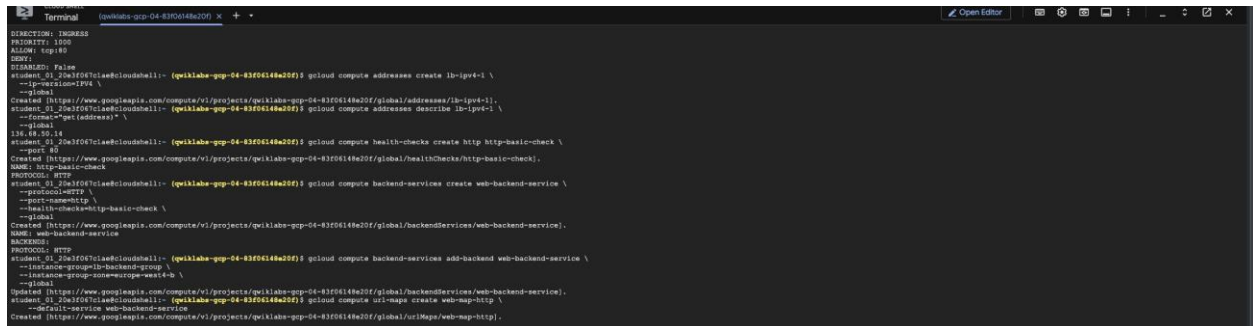


Figure 8. Backend Service

Step 8 – Configure URL Map

A URL Map was created to route incoming requests to the backend service.



```
student_01_20w3f0t1aa@cloudshell:~$ gcloud compute addresses create lb-ipvt-l \
--project=quiklabs-gcp-04-83f014ba20f \
--global \
Created [https://www.googleapis.com/compute/v1/projects/quiklabs-gcp-04-83f014ba20f/global/addresses/lb-ipvt-l].
student_01_20w3f0t1aa@cloudshell:~$ gcloud compute addresses describe lb-ipvt-l \
--project=quiklabs-gcp-04-83f014ba20f \
--global \
136.88.59.14
student_01_20w3f0t1aa@cloudshell:~$ gcloud compute health-checks create http-http-basic-check \
--port=80 \
Created [https://www.googleapis.com/compute/v1/projects/quiklabs-gcp-04-83f014ba20f/global/healthChecks/http-basic-check].
NAME: http-basic-check
PROTOCOL: HTTP
student_01_20w3f0t1aa@cloudshell:~$ gcloud compute backend-services create web-backend-service \
--protocol=HTTP \
--port=80 \
--health-check=http-basic-check \
--global \
Created [https://www.googleapis.com/compute/v1/projects/quiklabs-gcp-04-83f014ba20f/global/backendServices/web-backend-service].
NAME: web-backend-service
PROTOCOL: HTTP
student_01_20w3f0t1aa@cloudshell:~$ gcloud compute backend-services add-backend web-backend-service \
--instance-group=lb-backend-group \
--global \
Updated [https://www.googleapis.com/compute/v1/projects/quiklabs-gcp-04-83f014ba20f/global/backendServices/web-backend-service].
student_01_20w3f0t1aa@cloudshell:~$ gcloud compute url-maps create web-map-http \
--backend-service=web-backend-service \
Created [https://www.googleapis.com/compute/v1/projects/quiklabs-gcp-04-83f014ba20f/global/urlMaps/web-map-http].
```

Figure 9. URL Map Configuration

Step 9 – Create Target HTTP Proxy and Forwarding Rule

A Target HTTP Proxy and Global Forwarding Rule were configured for external traffic.

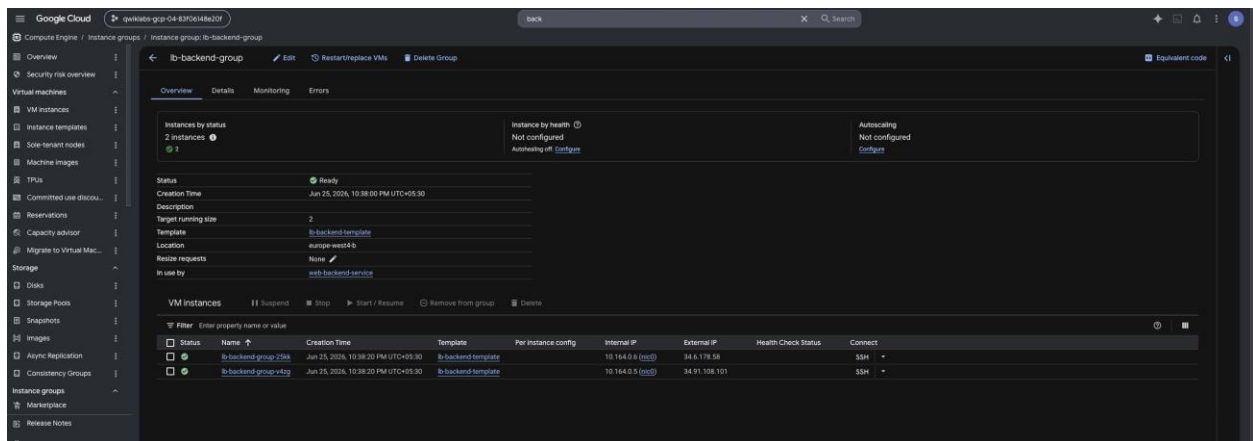


Figure 10. Forwarding Rule

Step 10 – Verify Load Balancer

The Application Load Balancer was successfully created.

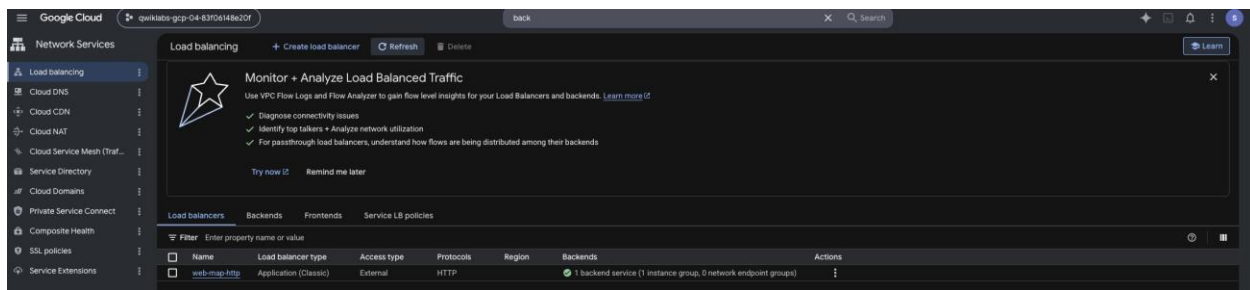


Figure 11. Load Balancer Overview

Step 11 – Verify Backend Health

The backend instances were verified as healthy using HTTP Health Checks.

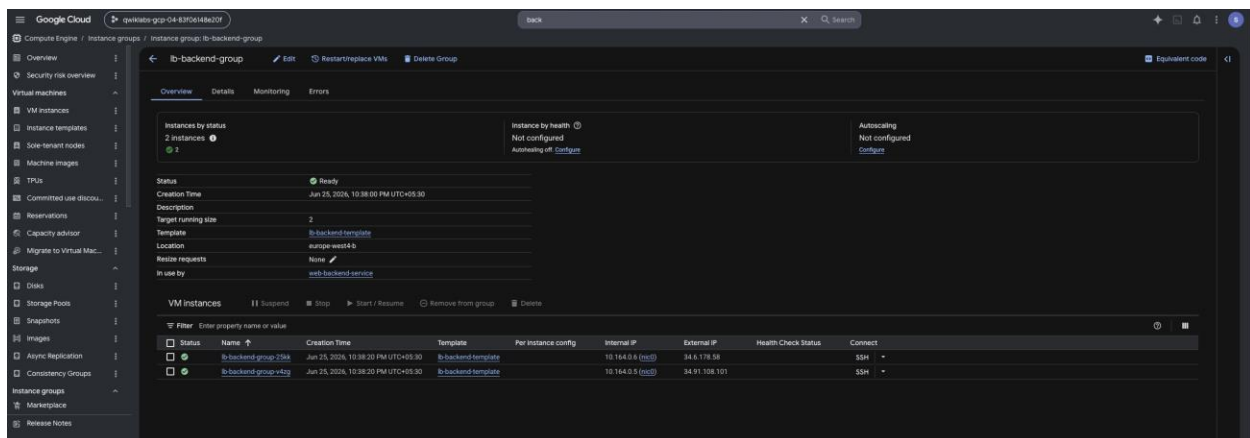


Figure 12. Backend Health Status

Step 12 – Verify Web Application

The external IP address of the Load Balancer successfully served the Apache web page.

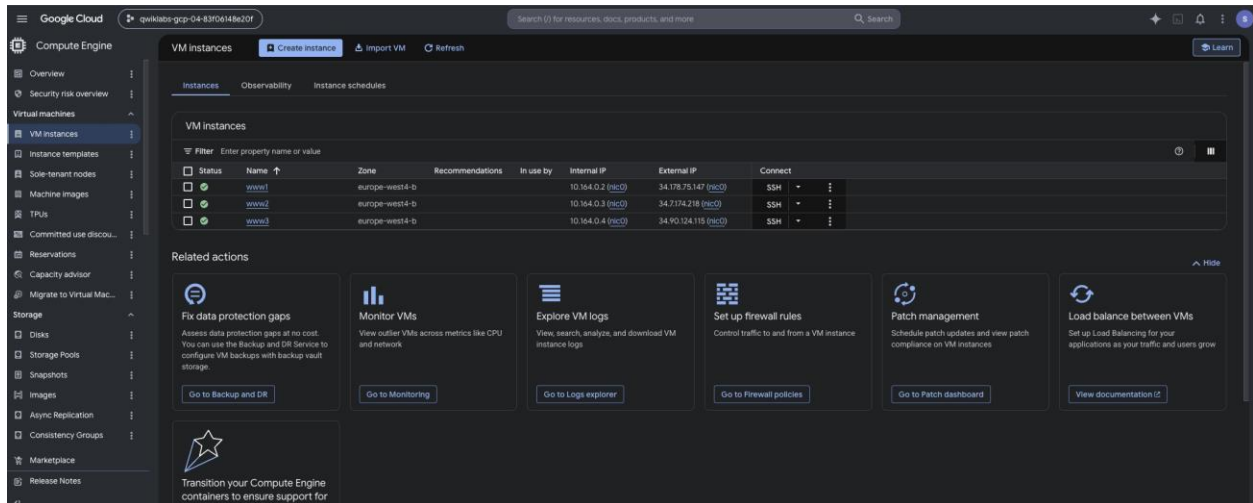


Figure 13. Browser Verification

6. Results

The Application Load Balancer was successfully deployed using Google Cloud Platform.

The implementation achieved the following:

- Successfully deployed three Compute Engine instances.
- Configured HTTP firewall rules.
- Created an Instance Template.
- Configured a Managed Instance Group.
- Configured HTTP Health Checks.
- Created Backend Service.
- Configured URL Mapping.
- Configured Target HTTP Proxy.
- Configured Global Forwarding Rule.
- Successfully verified healthy backend instances.
- Successfully served the application through the External HTTP Load Balancer.

7. Conclusion

This experiment demonstrated the deployment of a highly available web application using Google Cloud Application Load Balancer. By integrating Compute Engine, Managed Instance Groups, Health Checks, Backend Services, URL Maps, and Global Forwarding Rules, the application became scalable, fault tolerant, and capable of efficiently

distributing incoming HTTP traffic across multiple backend instances. This lab provided practical experience with one of the most commonly used production architectures in Google Cloud Platform.